FORGE — THE FIRST DESIGN-TIME ENGINE

Designer-First · Deterministic · Engine-Agnostic

01/ THE PROBLEM

Modern game development is increasingly consumed by complexity. As worlds grow larger and systems become more interconnected, design intent, content, implementation, tooling, and runtime behavior begin to drift apart.

The symptoms are familiar across the industry: designers script, technical designers glue systems together, engineers clean up logic, and artists rework assets after systems drift.

Teams compensate with custom tools, validation passes, spreadsheets, integration layers, and late-stage debugging. As complexity compounds, iteration slows and production becomes increasingly difficult to predict.

The result is: intent drift · simulation mismatch · narrative breakage · production churn · integration risk.

AAA development isn't constrained by talent. It's constrained by the growing cost of maintaining coherence across thousands of interconnected systems. That is the problem Forge was built to address.

02/ THE MISSING LAYER

Game engines solve runtime execution. They do not solve design-time coherence.

Studios have powerful runtimes but fragmented authoring environments. Designers think in systems. Engines think in implementation. The space between the two has become one of the largest sources of cost, risk, and production friction in modern development. **Forge was designed to fill that gap.**

03/ THE PLATFORM

Forge is a deterministic, engine-agnostic design-time layer that operates above Unreal, Unity, proprietary engines, and internal runtimes. It does not replace the engine. It standardizes how complex game systems are authored, simulated, validated, and delivered to the engine.

Designers author intent. Forge validates behavior. Engines execute outcomes.

Core Principles Author once · Simulate before production · Detect contradictions early	Preserve systemic coherence · Deploy deterministically · Reduce drift between design and implementation

Forge gives teams a unified environment for creating, visualizing, validating, and evolving complex game systems before those problems become production bottlenecks.

04/ THE IMPACT

When complexity becomes predictable: iteration accelerates, balancing becomes safer, content scales more efficiently, integration risk declines, and production becomes more resilient.

Forge is designed to reduce production churn, systemic rework, integration failures, and late-stage surprises that routinely consume months of development time on large projects.

On large AAA productions, these inefficiencies can represent tens of millions of dollars in schedule risk, lost development capacity, and preventable rework.

The result is not merely faster development.

It is greater creative freedom.

Studios spend less time fighting complexity and more time building experiences.

Forge — design-time engine for modern game development.
Working repository and full architectural brief available under NDA.
Elliott@Webb.Systems